

Advanced Cryptography

Elisabeth Oswald

Side Channels and Attacks

ROADMAP

Conceptual View

Side Channel Key Recovery Attacks

The (un)holy triad

Timing

Power and EM

Countermeasures

CRYPTOGRAPHY IS NOT THEORY

Making theoretically secure cryptography is not difficult:

- ▶ The one-time pad is perfectly secret, but also unusable in most contexts.
- ▶ A new secure cryptosystem is easily defined: just iterate a standardised cipher many times.

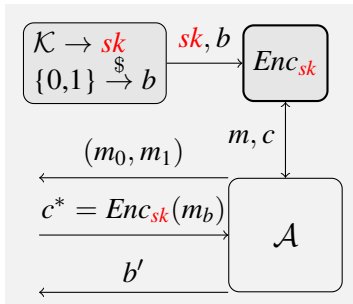
Cryptography is most interesting when we seek to find the “sweet spot” between theoretical guarantees and practical performance.

Can we be sure that “standard adversarial models” translate fully into practice? **The answer is mostly, but we found a major flaw in the late 1990s.**

THE POWER OF THE DARK SIDE

Theoretical cryptography: strong, but idealised assumptions about what an adversary gets in real life.

Implementing cryptography: dealing with the mess (leakage through power, EM, execution time, cache access, sound, noise, faults, ...).



(a) Indistinguishable Encryptions



(b) Key Recovery from Phone

ATTACKING IMPLEMENTATIONS OF CRYPTOGRAPHY

In the previous slide we assumed that the adversary has access to an encryption oracle.

In the real world this could be: a smart phone, a car key, a debit card, a credit card, an HSM in a cash machine, a web server, Alexa, Siri, etc.

The access to such oracles can be

- ▶ physical: via local access, typically for embedded systems
- ▶ logical: via an interface, either local or remote, the latter would be for cloud based systems

The access could be for a prolonged period of time, or short.

Sometimes adversaries can play with duplicates of the system: e.g. embedded devices, or via another account on the cloud service.

THE TRIAD: TIMING, POWER AND EM

With physical access one can do high precision timing, power and EM measurements.

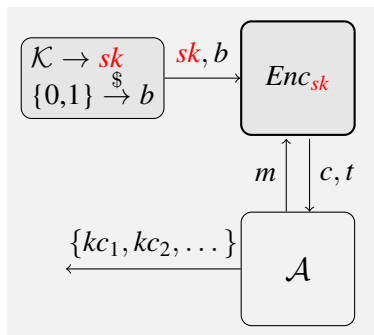
With logical access one can also record I/O data, including network traffic.

Within these channels, there is often a surprising amount of data dependency that relates to secret key usage or other secret data.

Attacks exploiting information from side channels have turned out to be extremely powerful in practice, and they concern **all** types of devices/systems.

We consider some in the context of embedded systems, with the aim of understanding how secret sharing plays a role in securing devices.

KEY RECOVERY VIA SIDE CHANNELS



Most side channel attacks have key recovery as a goal: divide and conquer independent chunks kc_i of the secret key.

The adversary supplies some input (m) in this illustration, and they may receive output (c in this example) and a side channel “trace”.

Univariate side channels (i.e. there is a single measurement, like overall execution time)

Multivariate side channels (i.e. instantaneous power/EM).

ROADMAP

Conceptual View

Side Channel Key Recovery Attacks

The (un)holy triad

Timing

Power and EM

Countermeasures

TIMING

Many optimisation techniques are based on exploiting data dependencies, which reveal information via side channels.

For instance: in implementations of RSA we use the **binary square and multiply** method. This is to optimise the runtime, and the conditional branch often opens up side channel attacks.

For instance: in some modern embedded processors, **multiplication can terminate early** if one of the multipliers is detected to be 0. This enables side channel attacks.

For instance: in modern processors with (shared) cache, processes can selectively **evict the cache**. The (re)loading time of variables depends if or not they are in the cache, and this enables side channel attacks.

TIMING ATTACK ON RSA

RSA

Given a public key pk and a secret key sk , with $pk = (e, N = pq)$ and $sk = d = e^{-1} \bmod \phi(N)$, the encryption of a plaintext m is given as $c = m^e \bmod N$. Decryption is given as $m = c^d \bmod (N)$.

RSA decryption is often implemented using a so-called square-and-multiply algorithm.

The modular multiplications are often implemented via the Montgomery method.

SQUARE AND MULTIPLY EXPONENTIATION

```
1 LeftToRight ( $N$ ,  $d = (d_w, d_{w-1}, \dots, d_1, d_0)$ ,  $c$ ) :  
2 begin  
3    $m = 1$   
4   for  $i = w - 1$  down to 0  
5      $m = m \cdot m \pmod{N}$  // (Montgomery multiplication)  
6     if  $d_i == 1$   
7        $m = m \cdot c \pmod{N}$  // (Montgomery multiplication)  
8   end  
9   return  $m$   
10 end
```

The algorithm computes $m = c^d \pmod{N}$ using the left-to-right binary algorithm.

The number of loop iterations is constant, but the number of conditional branches depends on the Hamming weight of d .

MONTGOMERY MULTIPLICATION

```
1 MontMult (x,y with  $0 \leq x,y < N$ ,  $N$ , int  $b$ ) :  
2    $r = 0$   
3   for  $i = 0$  upto  $n-1$  do  
4      $u = (r_0 + x_i \cdot y_0) \cdot \omega \pmod{b}$   
5      $r = (r + x_i \cdot y + u \cdot N)/b$   
6   end  
7   if  $r \geq N$  then  
8      $r = r - N$   
9   end  
10  return  $r$ 
```

The for-loop, and the various multiplications and additions, are in principle constant time.

The last if statement in the algorithm however introduces some data dependency in terms of execution time. It checks the condition $r \geq N$, and then performs an extra operation if necessary.

DIFFERENTIAL TIMING ATTACK: RECOVER d

- ▶ Record execution times of decryption operations for different inputs:
 $\{(c_i, t_i)\}, i \in \{1, \dots, N_t\}$.
- ▶ Predict behaviour of implementation by guessing unknown key bit by bit (divide and conquer principle).
- ▶ Assume bits d_w until d_{w-i} are known:
 - ▶ Guess bit $d_{w-(i+1)} = 0$, use a “model” to predict behaviour of MontMult decrypting the c_i at square in “next” loop iteration. If an extra operation occurs then add t_i to set s_0 otherwise add t_i so set s_1 . When all c_i have been processed compute the difference of average timings: $\delta_{w-(i+1)_0} = \bar{s}_1 - \bar{s}_0$.
 - ▶ Guess bit $d_{w-(i+1)} = 1$, use a “model” to predict behaviour of MontMult at square in “next” loop iteration. If an extra operation occurs then add t_i to set s_1 . When all c_i have been processed compute the difference of average timings: $\delta_{w-(i+1)_1} = \bar{s}_1 - \bar{s}_0$.
 - ▶ Select the guess that produces that bigger difference of averages.

WHY??

For a correct guess, the categorisation of timing values is correct for all ciphertexts.

Because an extra operation takes extra time, we expect the group of timing measurements corresponding to the extra operation gives a higher average than the group of timing measurements corresponding to no extra operation.

For an incorrect guess, the categorisation of timing values is only sometimes correct.

Therefore values that correspond to doing an extra step get mixed up with values that correspond to not doing an extra step, and thus, the averages of the two groups will be closer together.

ROADMAP

Conceptual View

Side Channel Key Recovery Attacks

The (un)holy triad

Timing

Power and EM

Countermeasures

DATA DEPENDENT POWER AND EM ILLUSTRATED ON INVERTER

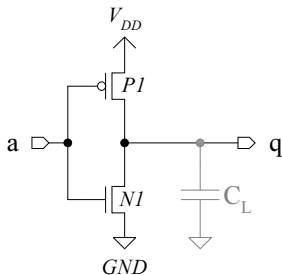


Figure: CMOS inverter.

Transition	Type of power consumption
$0 \rightarrow 0$	static
$0 \rightarrow 1$	static + dynamic
$1 \rightarrow 0$	static + dynamic
$1 \rightarrow 1$	static

Table: Output transitions of the CMOS inverter and the corresponding power consumption.

The inverter simply has two transistors $P1$ and $N1$.

In the inverter $P1$ is conducting and $N1$ is insulating if the input a is set to GND .

Vice versa, $P1$ is insulating and $N1$ is conducting if the input a is set to V_{DD} .

DATA DEPENDENT POWER AND EM ILLUSTRATED ON INVERTER

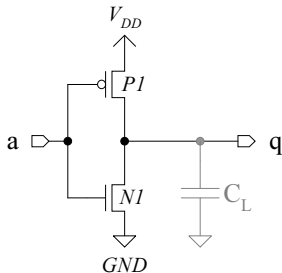


Figure: CMOS inverter.

Transition	Type of power consumption
$0 \rightarrow 0$	static
$0 \rightarrow 1$	static + dynamic
$1 \rightarrow 0$	static + dynamic
$1 \rightarrow 1$	static

Table: Output transitions of the CMOS inverter and the corresponding power consumption.

There is more current moving through the circuit when values change. This implies that the power consumption depends on the data.

Current flow changes the electromagnetic field, which implies that it is also data dependent.

This is a fundamental property of CMOS logic.

POWER ANALYSIS OF RSA DECRYPTION



Figure: RSA Correlation traces (taken from Messerges et al., 1999)

With the same idea as before: use a model to predict the behaviour of some aspect of the decryption operation for the “next unknown bit”.

Divide power traces into two sets and look at the difference of averages, or, use for instance a correlation score to decide about the key guess (see picture, bottom indicates correct bit guess).

ROADMAP

Conceptual View

Side Channel Key Recovery Attacks

The (un)holy triad

Timing

Power and EM

Countermeasures

THREE PRONGED APPROACH

Protocol level: change key often

Hiding: remove link between values stored in the device and leakage; can be done by modifying hardware or software.

Masking: remove link between predictable intermediate values and values that are actually held in the device; can be done in hardware or software.

Our next challenge is to understand why masking is the “go to” technique in practice.